



DESKTOP

Formation - Partie 3 - .NET

PLAN DE COURS

Chapitre 1 : Introduction à WPF

Chapitre 2 : Mise en page

Chapitre 3 : Contrôles et Data binding

Chapitre 4 : Architecture MVVM

Chapitre 5 : Persistence des données avec EF Core

Chapitre 6 : Intégration de services

1. Introduction à WPF

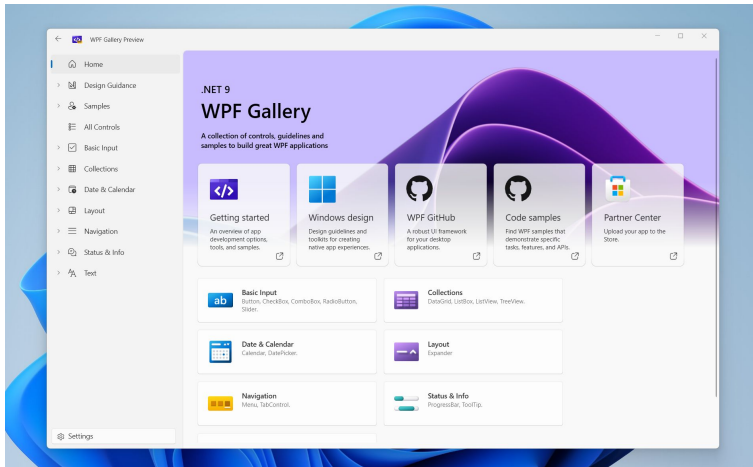
Qu'est ce que WPF

- Framework pour créer des applications desktop Windows **riches** et **performantes**
- Rendu via la carte graphique grâce à **DirectX**
- Unités de **mesure vectorielles** : l'interface reste nette, peu importe la taille de l'écran



Quand utiliser WPF ?

- Outil complexe exclusivement pour Windows
- Grande maturité logicielle
- Profiter de l'écosystème .NET



Séparation des préoccupations

- Distinction nette entre le design et la logique grâce au XAML
- C# : gère la logique métier, calculs etc...
- XAML : langage déclaratif basé XML pour définir l'UI



Anatomie d'un projet WPF

- App.xaml : Définition des ressources globales et point d'entrée.
- MainWindow.xaml : La fenêtre principale par défaut.
- Namespaces XAML : Importation des bibliothèques nécessaires (xmlns).

Le balisage XAML

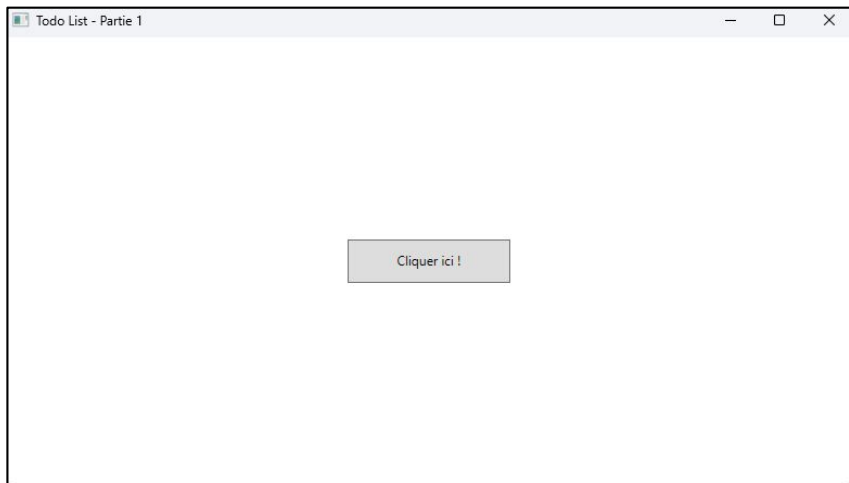
- Le XAML définit la hiérarchie visuelle.
- On utilise des **Grids** et des **StackPanels** pour le positionnement, et des **Styles** centralisés pour éviter la répétition.

```
XML

<Window x:Class="MonApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2000/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2000/xaml"
        Title="Ma Super App" Height="450" Width="800">

    <Grid>
        <StackPanel VerticalAlignment="Center" HorizontalAlignment="Center">
            <TextBlock Text="{Binding WelcomeMessage}"
                      FontSize="24" FontWeight="Bold"/>

            <Button Content="Cliquez ici"
                    Command="{Binding ClickCommand}"
                    Margin="0,20,0,0" Padding="10,5"/>
        </StackPanel>
    </Grid>
</Window>
```



Exercices

2. Mise en page

Jamais de coordonnées fixes

- Le desktop doit être responsive
- Nous utiliserons des conteneurs de disposition dynamiques :
 - [Grid](#)
 - [StackPanel](#)
 - [DockPanel](#)

Grid

- Système de lignes (**RowDefinitions**) et colonnes (**ColumnDefinitions**).
- Tailles : fixe (100), automatique (Auto), ou proportionnelle (*).
- Positionnement : Grid.Row="0" Grid.Column="1".



Définition de colonnes

```
<Grid.ColumnDefinitions>
  <ColumnDefinition Width="2*" /> <!-- Zone principale -->
  <ColumnDefinition Width="1*" /> <!-- Zone latérale -->
</Grid.ColumnDefinitions>
```

StackPanel et DockPanel

- **StackPanel** : Empile les éléments verticalement ou horizontalement.
- **DockPanel** : "Accroche" les éléments aux bords (Haut, Bas, Gauche, Droite).
- **WrapPanel** : Passe à la ligne si l'espace est insuffisant.

Exercices

3. Contrôles et Data binding

Les contrôles de base communs

- TextBlock pour l'affichage et TextBox pour une saisie
- Button, CheckBox, RadioButton
- ItemsControl pour les listes

Le Data binding

- Lien **automatique** entre une propriété C# et un élément UI
- Syntaxe : **{Binding NomDeLaPropriete}**
- **DataContext** : C'est l'objet que la vue regarde pour trouver ses données (pour que le XAML puisse accéder aux propriétés)

```
MainWindow.xaml.cs

namespace MonApp {
    public partial class MainWindow : Window {
        // 1. La propriété qui contient la donnée
        public string MonTexte { get; set; }
        public MainWindow() {
            InitializeComponent();

            // 2. LE PLUS IMPORTANT : On dit au XAML que
            // les données se trouvent dans cette classe
            this.DataContext = this;
        }
    }
}
```

```
MainWindow.xaml

<StackPanel Margin="20">
    <Label Content="Tapez quelque chose :" />

    <!-- CIBLE 1 : Le TextBox envoie la donnée vers la Source (C#) -->
    <!-- UpdateSourceTrigger=PropertyChanged : Envoie la lettre dès qu'on tape -->
    <TextBox Text="{Binding MonTexte, UpdateSourceTrigger=PropertyChanged}" />

    <Label Content="Affichage en temps réel :" Margin="0,20,0,0" />

    <!-- CIBLE 2 : Le TextBlock reçoit la donnée de la Source (C#) -->
    <TextBlock Text="{Binding MonTexte}" FontSize="20" Foreground="Blue" />
</StackPanel>
```

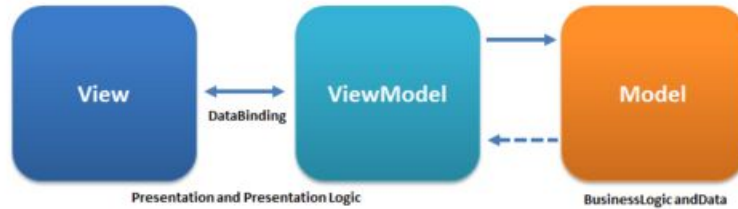
Exercices

4. Architecture

MVVM

Qu'est ce que MVVM

- **Model** : Les données (classe Note).
- **View** : L'interface XAML.
- **ViewModel** : Le cerveau qui prépare les données pour la vue et gère les actions.



L'interface INotifyPropertyChanged

- Indispensable pour prévenir la vue que la donnée C# a changé
- Sans elle, l'UI ne se met pas à jour si on modifie une variable par le code :



ViewModels/MainViewModel.cs

```
public class MainViewModel : INotifyPropertyChanged
```

Pourquoi MVVM ?

- Testabilité du code (Tests unitaires sans interface)
- Maintenance facilitée
- Spécialisation des tâches

Gestion des événements et commandes

- **Événement** : Click="Btn_Click" dans le code-behind (déconseillé en architecture pro car pollue le code qui gère l'affichage).
- **Commande** : objet C# qui encapsule l'action (meilleur pour le découplage).

Événement : peu recommandé

- Peu recommandé, sauf si l'action ne touche pas aux données mais seulement à l'apparence (couleur, animation, etc...)

```
MainWindow.xaml

<StackPanel Margin="20">
    <TextBox x:Name="InputNote" PlaceholderText="Écrire une note..." />

    <!-- L'événement est branché ici -->
    <Button Content="Ajouter" Click="BtnAjouter_Click" Margin="0,10" />

    <!-- On est obligé de donner un Nom (x:Name) pour y accéder en C# -->
    <ListBox x:Name="ListeNotesVisuelle" Height="100" />
</StackPanel>
```

```
MainWindow.xaml.cs

public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
    }

    // Cette méthode "pollue" le code-behind
    private void BtnAjouter_Click(object sender, RoutedEventArgs e)
    {
        // 1. Récupération manuelle de la donnée dans l'UI
        string texte = InputNote.Text;

        if (!string.IsNullOrEmpty(texte))
        {
            // 2. Manipulation directe d'un contrôle visuel (la ListBox)
            ListeNotesVisuelle.Items.Add(texte);

            // 3. Nettoyage manuel de l'UI
            InputNote.Text = "";
        }
    }
}
```


Exemple avec Commande

```
MainWindow.xaml

<StackPanel Margin="20">
    <!-- Binding sur la propriété TexteSaisi -->
    <TextBox Text="{Binding TexteSaisi, UpdateSourceTrigger=PropertyChanged}" />

    <!-- Binding sur la Commande -->
    <Button Content="Ajouter"
            Command="{Binding AjouterCommand}"
            Margin="0,10" />

    <!-- Binding sur la collection de données -->
    <ListBox ItemsSource="{Binding ListeNotes}" Height="100" />
</StackPanel>
```

```
MainWindow.xaml.cs

public partial class MainWindow : Window {
    public MainWindow() {
        InitializeComponent();
        // On branche le ViewModel
        this.DataContext = new MainViewModel();
    }
}
```

```
MainViewModel.cs

using System.Collections.ObjectModel;
using System.Windows.Input;
using Application_wpf.ViewModels; // Là où se trouve ta RelayCommand

namespace Application_wpf.ViewModel {
    public class MainViewModel {

        // La liste de données (se met à jour toute seule dans l'UI)
        public ObservableCollection<string> ListeNotes { get; set; } = new
        ObservableCollection<string>();

        // Propriété liée au TextBox
        public string TexteSaisi { get; set; }

        // La commande liée au Bouton
        public ICommand AjouterCommand { get; }

        public MainViewModel() {
            // Initialisation de la commande avec le RelayCommand
            AjouterCommand = new RelayCommand(execute => {
                if (!string.IsNullOrEmpty(TexteSaisi)) {
                    ListeNotes.Add(TexteSaisi); // On ajoute à la liste de données
                    TexteSaisi = string.Empty; // On vide la donnée (le TextBox suivra)
                }
            });
        }
    }
}
```

Exemple avec Commande

```
RelayCommand.cs

using System.Collections.ObjectModel;
using System.Windows.Input;

namespace Application_wpf.ViewModels {
    public class MainViewModel {
        public ObservableCollection<string> ListeNotes { get; set; } = new
        ObservableCollection<string>();
        public string TexteSaisi { get; set; } // Lié au TextBox

        // 1. On déclare la commande
        public ICommand AjouterCommand { get; }

        public MainViewModel() {
            // 2. On INSTANCIE la RelayCommand
            // Le premier bloc { } est l'action (Execute)
            // Le deuxième bloc (optionnel) est la condition (CanExecute)
            AjouterCommand = new RelayCommand(
                execute: (param) => {
                    if (!string.IsNullOrEmpty(TexteSaisi)) {
                        ListeNotes.Add(TexteSaisi);
                        TexteSaisi = string.Empty;
                    }
                },
                canExecute: (param) => !string.IsNullOrEmpty(TexteSaisi)
            );
        }
    }
}
```

Exercices

4. Persistence avec EF Core

Pourquoi utiliser EF Core avec WPF ?

- On utilise un ORM : on travail en C# pour les objets plutôt que directement du SQL
- On génère les tables à partir des classes (code first)
- Permet un suivi des modifications des objets pour simplifier les mises à jour



Les composants essentiels

- Le **DbContext** : la passerelle entre l'application et la base de données.
- Le **DbSet** : représente une table
- Le **Provider** : pilote spécifique à la base de données (SQLite, SQL Server, PostgreSQL)

```
TodoDbContext.cs

using Microsoft.EntityFrameworkCore;
using Partie5.Models;

namespace Partie5 {
    // Le DbContext est le pont entre notre application WPF et la base de données SQLite.
    // Il gère les connexions et convertit nos requêtes C# en requêtes SQL automatiquement.
    public class TodoDbContext : DbContext
    {
        // Tables de la base de données
        public DbSet<Tache> Taches { get; set; } = null!;
        public DbSet<Objectif> Objectifs { get; set; } = null!;

        // Méthode appelée pour configurer la base de données
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // On utilise SQLite. Le fichier 'todo.db' sera créé à la racine du dossier
            // d'exécution (bin/Debug/...)
            optionsBuilder.UseSqlite("Data Source=todo.db");
        }
    }
}
```

Cycle de vie des données

- **Migration** : Commande pour transformer vos classes C# en schéma de base de données :
 - a. Add-Migration InitialCreate
 - b. Update-Database

```
private void AjouterTache(object? parameter) {  
    var tache = new Tache {  
        Titre = NouvelleTacheTitre,  
        Categorie = NouvelleTacheCategorie,  
        Priorite = NouvelleTachePriorite,  
        DateCreation = DateTime.Now  
    };  
  
    // Ajout dans la base de données via EF Core  
    _dbContext.Taches.Add(tache);  
    _dbContext.SaveChanges(); // Persistance sur le disque !  
  
    // Ajout dans l'interface (mémoire)  
    Taches.Add(tache);  
  
    NouvelleTacheTitre = string.Empty;  
    NouvelleTacheCategorie = string.Empty;  
    NouvelleTachePriorite = "Normale";  
}
```

4. Intégration de services

Ajouter une couche de service

- Le ViewModel ne doit pas parler directement au DbContext
- On crée un Service pour centraliser les opérations CRUD
- Permet de faciliter les tests unitaires et séparer les responsabilités

```
private async Task AjouterTacheAsync() {  
    var tache = new Tache {  
        Titre = NouvelleTacheTitre,  
        Categorie = NouvelleTacheCategorie,  
        Priorite = NouvelleTachePriorite,  
        DateCreation = DateTime.Now  
    };  
  
    // Sauvegarde via le service asynchrone  
    await _todoService.AddTacheAsync(tache);  
  
    // Mise à jour de l'UI  
    Taches.Add(tache);  
  
    NouvelleTacheTitre = string.Empty;  
    NouvelleTacheCategorie = string.Empty;  
    NouvelleTachePriorite = "Normale";  
}
```

```
public async Task AddTacheAsync(Tache tache) {  
    using var context = new TodoDbContext();  
    context.Taches.Add(tache);  
    await context.SaveChangesAsync();  
}
```

Gestion de la concurrence

- WPF est **multithread** : l'interface est sur le thread principal : UI thread
- EF Core doit souvent être utilisé sur un thread différent pour faciliter la fluidité
- On utilisera **ToList()**, **Load()** ou **Local** pour la synchronisation des données

```
// Récupérer toutes les tâches sans figer la fenêtre
public async Task<List<Tache>> GetTachesAsync() {
    using var context = new TodoDbContext();
    return await context.Taches.ToListAsync();
}
```

Charger les données au démarrage

- Utiliser l'événement **Loaded** de la vue ou déclencher le chargement dans le constructeur de ViewModel

```
public partial class MainWindow : Window {
    public MainWindow() {
        InitializeComponent();
        // On affecte le DataContext ici plutôt qu'en XAML pour l'avoir sous la main
        facilement this.DataContext = new MainViewModel();
    }

    // Cette méthode est appelée quand la fenêtre a fini de se dessiner à l'écran
    private async void Window_Loaded(object sender, RoutedEventArgs e) {
        if (this.DataContext is MainViewModel viewModel) {
            // On déclenche le chargement asynchrone.
            // L'interface reste fluide pendant que la base de données travaille !
            await viewModel.LoadDataAsync();
        }
    }
}
```

```
<Window x:Class="Partie6.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:Partie6"
        xmlns:vm="clr-namespace:Partie6.ViewModels"
        mc:Ignorable="d"
        Title="Todo List - Partie 6 (Services & Async)" Height="500" Width="900"
        Loaded="Window_Loaded">
```

TP 3 : Validation des acquis